

PAXOS (Basic / Single-Decree)

Goal: Agree on one value. Fail-stop (not Byzantine). Persistent storage for fail-recovery.

Roles: Proposers, Acceptors, Learners (same node can play all).

Proposal #: [Round | ServerId] — totally ordered, globally unique.

Quorum: Any majority of acceptors; any two quorums share ≥ 1 node.

Phase 1 — Prepare/Promise:

- Proposer sends **Prepare**(n) to all acceptors.
- Acceptor replies **Promise**(n , (n_a , v_a)) if $n >$ any seen; promises *never to accept* $< n$; returns highest accepted (n_a , v_a) if any.

Phase 2 — Accept/Accepted:

- Proposer collects promises from majority; picks v = value from highest n_a (or own if none). Sends **Accept**(n , v).
- Acceptor accepts unless promised to higher n ; sends **Accepted**(n, v).
- Value *chosen* once majority accepted. “Accepted” $\neq$ chosen (single node).

Safety: Only one value chosen. **Liveness:** Not guaranteed — dueling proposers livelock. Fix: elect a distinguished proposer (leader).

Multi-Paxos: Stable leader skips Phase 1 for subsequent instances. Enables RSM. Used by Chubby, Spanner.

Cannot handle: Byzantine failures; stuck if no majority available (partition).

RAFT

3 states: Follower $\rightarrow$ Candidate $\rightarrow$ Leader; Leader sends heartbeats to Followers.

Terms: Monotonically increasing integer; each server stores **currentTerm**. Higher term always wins — stale leaders step down immediately.

Election:

- Follower timeout [$T, 2T$]; on expiry: increment term, become Candidate, vote for self, broadcast **RequestVote**.
- Grant vote only if: candidate log $\geq$ own (last entry term higher, or same term + longer); hasn’t voted this term.
- Win on majority; immediately send heartbeats (AppendEntries, no entries).

Log entry: (index, term, command). Index+term uniquely identify an entry (Log Matching Property).

AppendEntries RPC: (term, leaderId, prevLogIndex, prevLogTerm, entries[], leaderCommit). Follower rejects if log doesn’t match at prevLog*; leader backs up nextIndex[] and retries — discovers + fixes inconsistencies.

Committed = replicated to majority *AND* leader committed an entry from its *current term* (election safety: never count old-term entries alone).

Persistent state: currentTerm, votedFor, log[] — must survive crashes.

Volatile: commitIndex, lastApplied, nextIndex[], matchIndex[].

RequestVote RPC: (term, candidateId, lastLogIndex, lastLogTerm) $\rightarrow$ (term, voteGranted).

Leader Completeness: Elected leader has all committed entries — guaranteed by vote restriction (log must be at least as up-to-date).

CHUBBY

Goal: Coarse-grain *lock service* for loosely-coupled distributed systems. Provides advisory locks, small files for metadata.

Architecture: 5-server Paxos cell; one *master* handles all reads and writes (linearizable). Master lease prevents stale reads from non-masters. Clients cache file content + metadata.

Client sessions: Clients maintain KeepAlive lease with master. On session expiry, handles go *jeopardized* then unsafe. On master failure, clients wait for new master lease.

Handles: File/dir handles from **Open()**; hold locks, enable reads/writes. Locks are advisory. Events: lock acquired/released, file modified, master failover, handle invalidated — notified via callback.

Sequencer: Lock holder passes (lock-name, mode, seqno) to application servers; servers validate with Chubby master that lock is still valid. Prevents use of stale locks after failover.

Coarse-grained: Locks held for hours/days. Not for per-operation locking. Fat client library handles session management, caching, and failover.

Use cases: GFS master election, BigTable master lock, config metadata, distributed barrier.

Limitation: Single master = bottleneck at high RPC load; not for fine-grain locking.

ZOOKEEPER

Goal: General-purpose coordination kernel; clients build higher-level primitives.

Reads: From *any replica* — wait-free, may be stale (not linearizable). **Writes:** Through leader via ZAB protocol — linearizable, totally ordered.

Guarantees: Linearizable writes + FIFO per-client order = “A-linearizability”. sync() op forces read to see all prior writes.

Znodes: Persistent, ephemeral (deleted on client disconnect), sequential (auto-incremented suffix). Watches: one-shot async notification on change.

API: create, delete, exists, getData, setData, getChildren — all with optional watch.

Sessions: Client registers with a server; assigned session ID + timeout. Client must send heartbeats; on expiry all ephemeral nodes deleted. Session can migrate to another server on failure.

Versioned ops: setData(path, data, version) fails if stored version $\neq$ expected — compare-and-swap semantics. Enables safe conditional updates.

Performance: Read throughput scales linearly with replicas (any server).

Write throughput bottlenecked at leader (ZAB broadcast). Optimized for read-heavy coordination workloads.

Use cases: Leader election (ephemeral + watch), barriers, group membership, distributed locks (sequential znodes), config.

vs Chubby: ZK scales reads with replicas; Chubby does not. ZK = primitives; Chubby = locks. ZK trades stale reads for throughput.

EPAXOS (Egalitarian Paxos)

Goal: Leaderless consensus; optimal WAN latency; even load across all replicas.

Fast path (1 RTT): Non-conflicting commands — proposer sends **PreAccept** to $\lfloor F/2 \rfloor + \lceil F/2 \rceil + 1$ replicas; if all agree on dependency set, commit immediately.

Slow path (2 RTT): Conflicting (interfering, non-commutative) commands — falls back to standard Paxos Accept phase.

Agrees on: Command + *dependency set* (all cmds that must execute before it). Execution via dependency graph: find SCCs, topological sort, deterministic tie-break within SCC.

Instance space: Each replica r has its own column of slots $r.1, r.2, \dots$ — no global slot assignment. Any replica can propose in its own column without coordination.

Quorum sizes: $2F + 1$ replicas total. Fast quorum = $\lfloor 3F/2 \rfloor + 1$; slow (classic) quorum = $F + 1$ (majority).

Recovery: Crashed instances recovered via explicit Prepare phase (same as Paxos Phase 1) on the specific instance.

Interference: Cmds A, B interfere if $A; B \neq B; A$. Conflicting = interfering proposals at same replica in same instance.

vs Multi-Paxos: No single leader bottleneck; any replica proposes; natural for multi-datacenter. Tradeoff: more complex execution; slow path on conflicts.

CONSISTENCY MODELS (strong $\rightarrow$ weak)

Linearizability: Op appears atomic at a single point in real time between invocation and response. Non-overlapping: forced by real-time order. Concurrent (overlapping): any order valid. Single-copy semantics. (Raft, Chubby, Zab writes)

Sequential: Total order consistent with each client’s program order; no real-time constraint. (ZooKeeper overall, Lamport)

Causal: Causally-related ops seen in same order by all; concurrent ops may differ. (COPS, Bayou, Occult)

Eventual: All replicas converge if updates stop; no ordering. (Dynamo, Bayou reads)

Linearizability checking:

- Draw timeline: invocation/response intervals per op.
- Each op: pick atomic “execution point” within [*invoke, response*].
- Non-overlapping: execution order must match real-time order.
- Overlapping (concurrent): any ordering between them is valid.
- Linearizable iff valid assignment exists for all ops.

CAP: Under partition, choose Consistency (strong) OR Availability. Bayou/Dynamo = A; Raft/Chubby/Zab = C.

BAYOU

Goal: Weakly-connected replicated storage (mobile/intermittent); eventual consistency with app-controlled conflict resolution.

Write structure: Each write carries: *dependency check* (query: detect conflict), *merge procedure* (app-specific resolution code), *update*. If dep-check fails, merge proc runs.

Log: Tentative (unordered recent writes) + committed (stable ordered). Primary server assigns commit order. Tentative log re-executed as new writes arrive.

Write ID: (timestamp, serverId) — assigned at creation, globally unique, never changes. Used for ordering + deduplication during anti-entropy.

Anti-entropy: Pairwise gossip sync; nodes exchange missing writes by comparing version vectors. No central master required.

Session guarantee: Read-your-writes within a session (same server). Primary is the only server that commits writes and assigns stable sequence numbers.

Clients read tentative data — must tolerate uncertainty. On reconnect with primary, tentative writes become committed in primary’s order. CAP: chooses availability.

FACEBOOK TAO: EXISTENTIAL CONSISTENCY (SOSP ’15)

TAO: Facebook’s eventually consistent social graph cache. Objects = vertices (users, posts); edges = relationships. Async replication primary $\rightarrow$ replicas.

Vulnerability window: time during async replication when anomalies can occur.

Tension: Consistency (hard to quantify) vs Performance (simple to quantify). First study of consistency in a large-scale production system.

Anomaly types: (1) Real-time violation — stale read after write completes (OOB communication). (2) Ordering anomaly — two users see concurrent writes in different order (Oculus example).

Methodology: Sample 1-in-1M objects; log start/finish time, R/W, value. 12 days, 17M objects, 3B requests. Time skew P99.9 = 35ms — extend durations to avoid false positives. Local models (linearizability, per-object sequential, RAW) checkable per-object; non-local (causal) cannot be sampled.

Linearizability checker: Graph with write ops as vertices, real-time order as edges. Merge read with its write. *Anomaly iff cycle*.

Results: 5 linearizability anomalies/million reads (TAO is highly consistent).

Sensitive apps mark reads *critical* → routed through master → linearizable.

RIFL (Reusable Infrastructure for Linearizability)
Problem: RPCs are at-least-once — retries re-execute non-idempotent ops (double-charge, double-insert).
Solution: Exactly-once RPCs. Tag each RPC with (clientId, rpcSeqNum). Server stores result durably; on retry returns stored result, skips re-execution.
ClientId: Assigned by master with lease. Lease expiry + no renewal ⇒ safe to GC stored results.
RAMCloud impl: Results stored in log alongside data; GC when client ACKs receipt. < 5% overhead; ≈22 μs transaction commit latency.

OCCULT

Problem — Slowdown Cascade: In causal consistency with heterogeneous servers: client reads from slow replica S_1 → picks up old timestamp → subsequent reads from faster replicas S_2, S_3 must wait until they’re as fresh as S_1 → cascade of waits across all future reads.
Occult (“I Can’t Believe It’s Not Causal!”): Break cascade by identifying writes that are *nil-external* at the slow replica. If the causal dependency is nil-external, a faster replica can skip waiting for it — client observes same result. Achieves near-linearizable performance while preserving causal consistency guarantees. (SOSP ’21)
Setting: Heterogeneous cluster — mix of fast (SSD) and slow (HDD) replicas all claiming causal consistency. One slow read poisons all subsequent reads across faster replicas.
Protocol: Annotate ops as nil-external when provably unobservable. Receivers skip waiting for nil-external deps in the causal chain — cascade broken.
Result: 60–90% latency reduction vs standard causal consistency; performance indistinguishable from linearizable for most workloads.
Key insight: Not all causal dependencies are observable. If dependency is invisible to client, enforcing it is unnecessary cost.

NIL-EXTERNALITY

Nil-External (NIL) op: An operation whose effects *cannot be observed by any external agent* at that instant. Example: write to a key that no current client is watching; read of a value that will be overwritten before anyone sees it.
Exploitation: Nil-external ops can be deferred, reordered, or batched without violating correctness from any client’s perspective.
Application: Used by Occult to break slowdown cascades. Used more broadly to reduce coordination costs in distributed systems — skip expensive synchronization when the operation’s effects are provably invisible.

GFS (Google File System)

Architecture: 1 Master (all metadata in memory; op log + checkpoints) + many Chunkservers (64 MB chunks, 3 replicas as Linux files).
Metadata: File namespace, file→chunk map, chunk→location. Locations *not persisted* — re-pollled at startup/heartbeat.
Read: Client asks master for chunk handle + locations → reads from closest chunkserver. Master *not on data path*.
Write (mutation):
1. Client gets primary + secondaries from master (60s lease to primary).
2. Client pushes data in chain to all replicas (network-topology aware, buffered in LRU).
3. Client sends write to *primary*; primary assigns serial order, applies, forwards to secondaries.
4. Secondaries ACK to primary; primary ACKs client. Any failure → retry.

Record Append: Atomic append at GFS-chosen offset (last chunk). Padding if overflow. At-least-once (client retries may get duplicates).
Consistency: Relaxed/weak — defined but not consistent after failed mutations. Non-POSIX. High sustained bandwidth > low latency (design priority).
Caching: Clients and chunkservers do *not* cache data. Clients cache metadata (chunk locations) only.
Metadata: Namespace + file→chunk mapping persisted (op log); chunk→location *not* persisted — re-pollled via heartbeats. Heartbeats: chunkserver → master.

Locking: Read locks on all parent directories; write lock only on the target file. Directories have no metadata.
Deletion: Lazy — file renamed to hidden name; master GCs after fixed interval. Heartbeat replies list chunks no longer in master metadata → chunkserver deletes them.
Chunk size 64 MB: Reduces master interactions (fewer metadata lookups), reduces metadata size, enables persistent TCP connections. Downside: hot spots for small popular files.
Master in RAM: All metadata fits in memory; master not on data path after initial chunk lookup — clients go directly to chunkservers.
Stale detection: Chunk version numbers — master detects stale replicas after chunkserver restart. **Shadow masters:** read-only replicas of master state (slightly stale). **Checksums:** per 64 KB block to detect corruption.

DYNAMO (Amazon)

Requirements: ~10k servers; P99.9 latency target; *always-writable* — highly available writes over strong consistency (shopping cart).
Consistent hashing: Keys + nodes on virtual ring; key owned by first N clockwise nodes. *Virtual nodes:* each physical server = multiple ring tokens (load balance, heterogeneity).
Preference list: First N distinct physical nodes clockwise; sloppy quorum

uses these.

Vector clocks: Per-key causal history: [(node, counter)]. Client reconciles divergent versions (siblings) on read. Pruning: drop oldest if clock too large.
Sloppy quorum: $R + W > N$ (typically $N=3, R=2, W=2$). “Sloppy” = any N healthy nodes, not strictly the ring’s N . *Hinted handoff:* write to substitute with hint to forward on recovery.
Anti-entropy: *Merkle trees* per key range — efficient diff detection; background sync of diverging subtrees only.
Gossip: Failure detection + membership — no central coordinator.
get()/put() context: Both return a *context* object containing the version vector [(node, counter)] for all nodes a write reached. Client passes context back on writes to maintain causality.
CAP choice: Explicitly AP — during partitions, writes always succeed on any healthy node; consistency deferred to read time via reconciliation.
Read repair: After client reconciles divergent siblings, merged version written back to out-of-date replicas.
Reconciliation: Syntactic (automatic — e.g., last-write-wins by timestamp) vs Semantic (app-defined — e.g., union of cart). Syntactic can lose data.

BIGTABLE

Goal: Semi-structured data at petabyte scale: 100 TB+, billions URLs, few-ms latency, variable schema.
Data model: Sorted map: (rowkey, col_family:qualifier, timestamp) → value. Rows sorted lexicographically. Row range = *tablet* (distribution + load balancing unit).
Architecture: Master (tablet assignment, load balancing, schema, GC) + Tablet Servers (serve ~100 tablets, R/W) + GFS (SSTables + WAL) + Chubby (master election, tablet server liveness, schema locks, root tablet location).
Write: WAL + memtable. **Read:** Merge memtable + SSTables (most recent wins).
Compaction: Minor (memtable → SSTable), merging (multiple → one), major (all merged, deletes applied). Bloom filters on SSTables for fast negative lookups.
Locality groups: Column families grouped on disk; separate SSTables per group; mark group in-memory for hot data; compression per-group.
Row atomicity: Reads/writes to a single row are atomic across all column families — strongest per-row guarantee.
Wide vs Tall rows: Wide = many columns per row (“tell me everything about this key”, <100 MB/row). Tall = split into multiple rowkeys with field promotion (“does relationship exist?”). Wide adv = one lookup; Tall adv = efficient range scans on sub-entities.
API: Write: **set/delete** — atomic per-row. Read: **lookup** (point) or **stream** (scan range across column families) — more efficient, fewer round-trips than key-value.
Client caches tablet location after first lookup — bypasses master on subsequent requests. Master is thin (not on data path).
Consistency: Strongly consistent within a tablet — single tablet server at a time (Chubby ensures). Not eventual.
vs GFS: unstructured bytes. **vs RDBMS:** too large, no fixed schema. **vs Dynamo:** stronger consistency.

SPANNER (Google)

Goal: Globally-distributed, externally consistent (linearizable), cross-shard ACID transactions, SQL.
Motivation: BigTable lacks general-purpose transactions, complex multi-table schemas, SQL. Spanner unifies at global scale.
Deployment: Universe → Zones (datacenters) → Spanservers. One *Paxos group per shard (tablet)*; elected leader per group; *directory* = smallest unit of data movement.
TrueTime API: TT.now() → interval [earliest, latest]; TT.after(t), TT.before(t). GPS + atomic clocks per datacenter. Uncertainty $\epsilon \approx 1\text{--}7$ ms; bounded.
External consistency: If T_1 commits before T_2 starts $\Rightarrow s_1 < s_2$. Achieved by *commit-wait*: pick $s \geq TT.now().latest$; wait until $TT.after(s)$ before releasing locks.
Read-only txn: Snapshot read at t_{safe} (all in-flight writes $\leq t$ applied). No locks; any up-to-date replica. Timestamp = $TT.now().latest$.
Read-write txn: 2PL (acquire locks) + 2PC (cross-shard atomic commit); each Paxos group logs prepare/commit. Commit-wait ensures external consistency.
Globally consistent reads @ timestamp: Any replica can serve a read-only txn at a past timestamp t once $t_{safe} \geq t$ — enables consistent snapshots across shards without locks.
 $t_{safe} = \min(t_{safe, Paxos}, t_{safe, TM})$. Paxos safe time = timestamp of highest prepared write. TM safe time = ∞ if no in-flight 2PC transactions (else blocked). Read-only txn at t must wait until $t_{safe} \geq t$.
Strict serializability: Spanner achieves serializability (txn isolation) + external consistency (real-time ordering) = strictest isolation. Wound-wait used for deadlock avoidance.
Schema: Relational tables; *interleaved* tables store child rows adjacent to parent — eliminates join overhead for hierarchical data.
Challenges without TrueTime: (1) Non-leader replicas lag → stale reads. (2) Cross-shard txns need atomic commit across independent Paxos groups. TrueTime solves ordering; 2PC solves atomicity.